

ScanCoder: Leveraging Human Attention Patterns to Enhance LLMs for Code

ANONYMOUS AUTHOR(S)

Code comprehension is a fundamental challenge in software engineering that impacts developer productivity and software quality. While Large Language Models (LLMs) demonstrate strong capabilities in code generation and summarization, they process code differently from human developers, who employ strategic attention patterns focused on semantically critical elements. Recent research has successfully integrated human attention patterns captured through eye-tracking into AI models for software engineering tasks, however, existing human-AI approaches face critical limitations that prevent widespread practical deployment, particularly for LLM enhancement. Existing approaches to incorporate human cognitive insights face scalability limitations due to resource-intensive eye-tracking studies and lack empirical validation for cross-language generalizability.

We present *ScanCoder*, a framework that integrates cognitive simulation with LLM enhancement through (1) generating human-like attention patterns at scale using minimal eye-tracking data via cognitive simulation with Adaptive Character of Thought-Rational (ACT-R) architecture, and (2) cognitively-guided fine-tuning that emphasizes tokens according to their cognitive salience and attention order. Our approach demonstrates cross-language transfer by applying C++-derived cognitive patterns to enhance Java programming tasks. Comprehensive evaluation on CodeXGLUE benchmarks shows consistent improvements across different LLM architectures, achieving gains of 39% for CrystalBLEU completion metrics and 22% for BERTScore summarization. Mechanistic analysis reveals that cognitive guidance reshapes model attention in task-dependent ways, increasing focus on semantically critical tokens by 2.5×. This work establishes the first scalable framework for integrating simulated human cognitive patterns into LLM training, enabling more interpretable and effective code understanding.

CCS Concepts: • **Software and its engineering** → *Software maintenance tools*.

Additional Key Words and Phrases: Human-guided AI, Eye Tracking, Code Comprehension,

ACM Reference Format:

Anonymous Author(s). 2025. ScanCoder: Leveraging Human Attention Patterns to Enhance LLMs for Code. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 21 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Large Language Models (LLMs) have revolutionized software engineering by demonstrating remarkable capabilities in code generation, completion, and summarization [17, 24, 28, 58]. However, unlike human developers who exhibit systematic cognitive strategies when reading code—such as following control flow, identifying key variables, and understanding semantic relationships—LLMs operate as black boxes, making it difficult to understand whether their internal representations align with human-like code comprehension strategies [8, 9, 20, 38, 68]. These models face interpretability challenges in understanding how they process and comprehend code structures [7, 40, 60]. This gap between human cognitive processes and machine learning approaches presents both an opportunity

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

and a challenge for developing more interpretable and effective AI systems for software engineering tasks [44, 49, 56].

Human-AI collaboration in software engineering has increasingly focused on leveraging human cognitive processes to enhance artificial intelligence capabilities and interpretability [2, 12, 43, 53]. Eye-tracking has emerged as the predominant approach for capturing attention patterns from developers during code reading tasks [26]. These empirical studies consistently demonstrate that expert programmers exhibit systematic attention behaviors, focusing on semantically critical code elements, following logical execution flows, and employing predictable reading strategies that correlate with task complexity and expertise levels [10, 30, 37, 45]. Building upon these insights, recent research has successfully integrated human attention patterns into AI models for software engineering tasks [4, 36, 66, 67]. Neural networks aligned with human gaze patterns achieve improved code comprehension difficulty prediction [4], while attention mechanism fusion enhances neural code summarization performance [67]. Previous research has also established that semantic abstractions across different programming languages share common cognitive structures [10, 45], providing theoretical foundations for potential cross-language applications.

Despite these advances, existing human-AI approaches face multiple limitations that prevent widespread practical deployment. First, these approaches exhibit scalability constraints, with eye-tracking studies typically recruiting only 15-40 participants due to resource limitations and requiring substantial time investments for data collection [33, 54]. Second, laboratory-controlled experimental settings compromise the ecological validity of collected attention data [43]. Third, although theoretical foundations suggest cross-language cognitive similarities, existing human-AI approaches operate within single programming languages, leaving cross-language generalizability empirically unvalidated [10, 45]. To address data scarcity, current solutions resort to either data augmentation techniques that generate unrealistic attention patterns [41, 67], or constraining applications to small-scale, simplified tasks [4]. These combined limitations create a gap between demonstrated potential and practical deployment requirements for large-scale LLM model training.

Our ScanCoder framework addresses these limitations through a novel cognitive simulation approach integrated with LLM enhancement. To overcome scalability constraints, we leverage CodeACT-R [64], a cognitive simulation model based on ACT-R architecture [5, 48], which generates human-like attention patterns at scale using minimal initial eye-tracking data. To validate cross-language generalizability, the original CodeACT-R is designed on C++ comprehension data while applying the resulting attention patterns to improve LLM performance on Java programming tasks. We provide the first empirical validation of cross-language attention pattern transfer. Most importantly, we demonstrate effective integration of these simulated attention patterns into LLM training for software engineering tasks, which represents the first successful application of cognitively-simulated human attention in LLM enhancement. This comprehensive approach provides a scalable, cost-effective way to conduct Human-LLM improvement.

As shown in Figure 1, the ScanCoder framework encompasses two main components that work synergistically to enhance LLM performance: (1) CodeACT-R simulation that generates human attention pattern (scanpath) to capture semantic comprehension strategies from collected eye-tracking data, and (2) cognitively-guided fine-tuning that systematically integrates these simulated attention patterns into LLM training by emphasizing tokens according to their cognitive salience. We evaluate our approach on CodeXGLUE benchmark tasks, demonstrating consistent and performance improvements across both code completion and summarization tasks on LLM architectures [39]. Through attention mechanism analysis, we reveal task-dependent reshaping of model focus, where our method alters how models process code by adopting human-like attention strategies. We hope our approach can create a new paradigm for incorporating human cognitive processes into AI

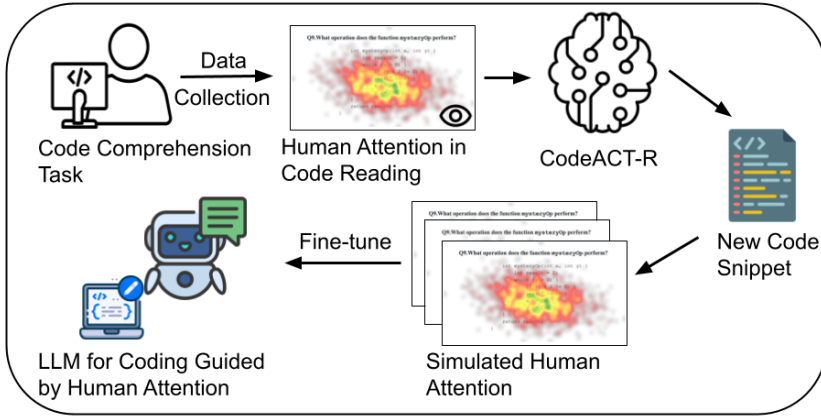


Fig. 1. Overview of the ScanCoder framework. The framework transforms human attention patterns collected during code comprehension tasks into cognitive guidance for LLM enhancement. Starting from CodeACT-R simulates human attention patterns (scanpath), which are then integrated into LLM training through cognitively-guided fine-tuning to produce enhanced models for software engineering tasks.

model development, offering both theoretical insights into human-machine alignment and practical solutions for resource-constrained cognitive data collection in software engineering research.

We summarize the contributions of this paper as follows:

- (1) We present **ScanCoder**, a comprehensive framework combining cognitive simulation through ACT-R architecture with LLM enhancement, offering the first scalable solution for integrating human attention patterns into LLM Model training for software engineering.
- (2) We demonstrate cross-language generalizability of human attention patterns, showing that semantic abstractions transfer effectively across programming languages, enabling models designed on C++ cognitive data to improve Java programming task performance.
- (3) We achieve performance improvements on software engineering tasks, with gains reaching 39% for completion metrics and 22% for summarization across LLM architectures.
- (4) We provide mechanistic insights into how cognitive guidance reshapes LLM attention, revealing task-dependent strategies where models learn focused local patterns for completion and distributed patterns for summarization, increasing attention on semantically critical tokens by 2.5×.

The remainder of this paper is organized as follows: Section 2 demonstrates the motivating example, Section 3 presents the background and related work; Section 4 introduces the detailed implementation of CodeACT-R; Section 5 introduces the methodology of ScanCoder; Section 6 describes our experimental setup; Section 7 presents the evaluation results; Section 8 and 9 discuss the implications and limitations of the proposed work, and Section 10 concludes the paper.

2 Motivating Example

Large Language Models (LLMs) have achieved success in software engineering tasks, yet they differ from human developers in how they process and comprehend code structures. While LLMs process tokens sequentially from left to right, humans exhibit sophisticated, strategic attention patterns that selectively focus on semantically critical elements while intelligently skipping redundant syntactic constructs [10, 65]. This divergence between machine and human code comprehension strategies

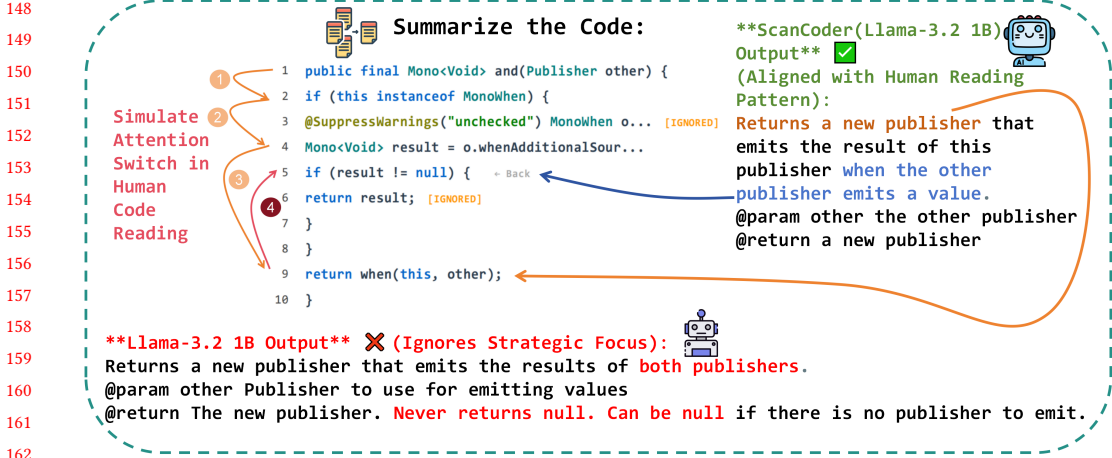


Fig. 2. Motivating example in CodeXGLUE demonstrating strategic human attention patterns in code reading. ScanCoder leverages human-like attention (attending to 5 of 10 lines) to produce accurate summaries, while baseline models without cognitive guidance generate incorrect outputs with logical contradictions.

presents both a challenge and an opportunity: can we leverage human cognitive patterns to guide LLMs toward more effective code understanding?

2.1 Strategic Human Code Reading: An Example

To illustrate this phenomenon, consider the Java method shown in the uploaded figure, which demonstrates how ScanCoder leverages strategic human attention patterns. This method combines two reactive publishers, and our CodeACT-R simulation reveals that human programmers exhibit a strategic gap pattern when reading this code. The programmers attend to only 5 out of 10 lines while maintaining complete logical understanding. The scanpath follows the sequence [line 1, line 2, line 4, line 9, line 5], strategically skipping others lines.

Critically, this non-sequential attention pattern demonstrates sophisticated cognitive efficiency rather than random skipping. Humans strategically: (1) *Skip predictable syntax*—after encountering the conditional return statement, they skip the obvious closing brace because its presence is syntactically predictable; (2) *Ignore semantic-free content*—they skip lines that contain no semantic information; (3) *Maintain logical coherence*—despite the gaps, they attend to both execution paths: the optimized path and the fallback path.

This strategic reading pattern leads to better code comprehension outcomes. When asked to summarize this method, our attention-aligned model (ScanCoder) produces a summary that accurately captures the method’s dual-path logic and abstraction level, demonstrating that strategic attention patterns facilitate appropriate abstraction—focusing on functional behavior rather than implementation details. In contrast, baseline models without strategic attention guidance produce incorrect outputs. Such outputs contain logical contradictions (“Never returns null. Can be null”) and miss the strategic conditional logic that the human attention pattern successfully identified.

2.2 The Opportunity: Guiding LLMs with Human Attention

The key insight driving our work is that human attention patterns encode semantic prioritization strategies learned through years of programming experience. These patterns represent not just where humans look, but what information is most relevant for code understanding tasks. By

incorporating these cognitive signals into LLM training, we hypothesize that models can learn to: (1) prioritize semantically critical tokens over syntactic boilerplate; (2) develop task-appropriate attention strategies (local focus for completion, global focus for summarization); and (3) achieve more confident and accurate predictions by following cognitively-informed reading flows.

The motivation shows potential to bridge two gaps in current LLM for software engineering: (1) the interpretability between human cognitive processes and LLM internal representations, and (2) the scalability between the need for large-scale cognitive data and the practical limitations of human studies.

3 Background and Related Work

This section reviews three research areas: eye-tracking methodologies in software engineering that capture human cognitive processes during code comprehension, cognitive simulation approaches using computational architectures like ACT-R to model human behavior, and human-enhanced AI systems that successfully integrate human insights into deep learning.

3.1 Eyetracking in Software Engineering

In recent years, Eye-tracking has become a widely-used tool in software engineering to collect human study for different purposes [3, 23]. Eye-tracking research in programming has been applied to areas such as program comprehension and debugging. Researchers use various eye-movement metrics, such as fixation duration and saccades, to obtain quantitative insights into how developers understand code, navigate through programs, and detect bugs [43]. The eye-tracking data provide insights into cognitive processes during software engineering tasks, such as code comprehension and debugging. It helps researchers understand how developers interact with code and other software artifacts, offering a deeper understanding of software engineering practices and challenges [54].

The eye-tracking technology is also instrumental in educational contexts, enhancing the understanding of how novices learn to read and understand programs, and identifying improvements in teaching and tools [37, 45]. This cognitive process sheds light on eye-tracking for novice programming, aiding programmers in improving their skills. Human attention data assists deep learning models in improving performance on programming tasks such as code comprehension [4, 67]. This preprocessing ensures that the attention patterns correspond precisely to the meaningful code tokens used for model learning.

3.2 Cognitively Simulating Human Behavior

Using cognitive methods to simulate human behavior involves the development and application of computational neural models that mimic cognitive processes underlying human actions and decisions.

A hybrid model combining cognitive processes and a physical human motor framework has been utilized to develop a virtual driver model [22]. Researchers propose leveraging cognitive processes to generate data for pretraining reinforcement learning models, thereby reducing the necessity for repeated user interactions [62]. In the realm of interface testing within human-computer interaction, the Cognitive Model Interface Management System (CMIMS) enables cognitive models to interact with interfaces akin to users [47]. Moreover, in human-agent interaction, cognitive models facilitate pretraining, adaptive interaction, and the generation of personalized behavior for artificial agents [51]. These methods offer insights into simulated human behavior in various fields, including human-robot interaction, driving simulation, and cognitive neuroscience.

Besides, as an important computational cognitive model, the ACT-R framework has been employed to simulate visual attention in specific tasks, such as random menu selection task [27]. Byrne

and Anderson demonstrated that the ACT-R effectively fits data, enhancing our understanding of menu selection and visual search processes [11]. Additionally, Salvucci and Gray have shown that the ACT-R can simulate driver gaze distributions during lane keeping, curve negotiation, and lane changing [50]. In summary, the ACT-R has been extensively used for simulating human cognitive processes. It can adapt to different environments and accurately mimic human behavior.

3.3 Human-Enhanced AI Systems

Human-enhanced AI represents a paradigm where human knowledge, expertise, and feedback are systematically integrated into AI systems to improve their performance, robustness, and alignment with human values [25, 42]. This approach encompasses a broad spectrum of methodologies that leverage human cognitive capabilities to augment machine learning processes, ranging from data annotation and active learning to sophisticated feedback mechanisms that guide model training and inference [1, 34]. Human-enhanced AI recognizes the unique value of human insight, creativity, and domain expertise in addressing complex real-world problems.

Contemporary research identifies three primary categories of human involvement: improving model performance through enhanced data processing, interventional model training that integrates human feedback during the learning process, and system-level human-in-the-loop designs that maintain human oversight throughout deployment [52, 59]. Active learning represents a foundational approach where AI systems strategically query human experts for labels on the most informative examples [32]. Interactive machine learning extends this concept by enabling continuous collaboration between humans and learning systems, while machine teaching frameworks position human domain experts as instructors who control the learning process through curriculum design and strategic example selection [29]. Reinforcement Learning from Human Feedback (RLHF) has emerged as a particularly influential paradigm within human-enhanced AI [6, 16, 18]. This approach has been instrumental in developing sophisticated AI systems including ChatGPT, InstructGPT, and Claude, where human evaluators provide preference feedback that shapes model behavior toward more helpful responses [13]. Recent advances in RLHF have addressed challenges including sparse and noisy feedback signals, demonstration of cross-domain generalizability, and development of more efficient preference collection methodologies [31, 61].

The practical applications of human-enhanced AI span diverse domains including autonomous systems, natural language processing, computer vision, and software engineering [21, 57]. In autonomous navigation and robotics, human-in-the-loop approaches enable dynamic adaptation to complex environments through real-time feedback and intervention mechanisms [14]. For natural language processing, it has become the gold standard for training conversational AI systems that exhibit appropriate social behaviors and ethical reasoning [15]. As AI systems become increasingly capable and deployed in high-stakes applications, the systematic integration of human knowledge and values through human-enhanced AI approaches becomes essential for ensuring safe, reliable, and beneficial AI systems that serve human needs and societal goals [35, 55].

4 CodeACT-R: Human Attention Pattern Simulation

This section presents the methodology for generating human-like attention patterns through CodeACT-R simulation [64]. We adapt the original CodeACT-R model, initially developed for C++ code comprehension, to generate scanpaths for Java programming tasks, thereby establishing cross-language generalizability of cognitive attention patterns.

4.1 Scenario Description

The CodeACT-R model aims to simulate scanpaths during code reading at the statement level. A scanpath is the sequence of statements a programmer fixates on, recorded in order. The CodeACT-R

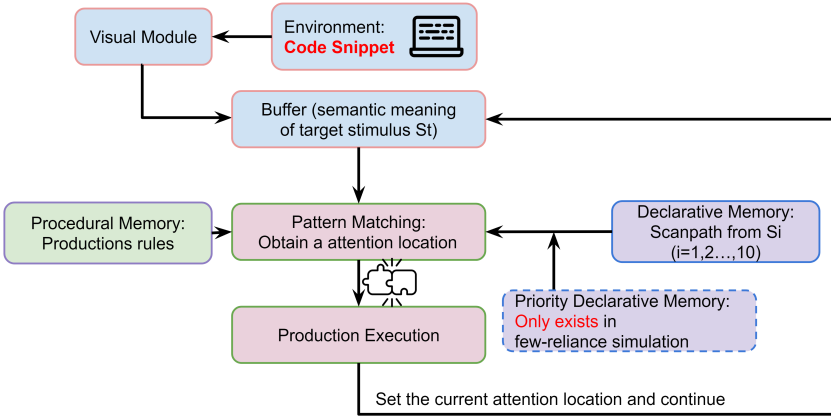


Fig. 3. The architecture of the CodeACT-R.

collected initial eye-tracking data from participants with various programming skills in performing with code comprehension tasks in C++. The CodeACT-R collects all main types of control symbols in C++ (e.g., if, while, for, switch) to achieve a comprehensive understanding of the semantic meanings of the words.

4.2 The Components of CodeACT-R

In Figure 3, we present the CodeACT-R architecture used in our study. The components of the architecture include:

- **Buffer:** The CodeACT-R encodes the target code snippet from the environment and captures the semantic meaning, which is then stored in the ACT-R buffer.
- **Declarative Memory:** This is the knowledge base of the model, containing the semantic information and corresponding scanpaths collected from human participants.
- **Production Rule:** Production rules perform pattern matching between the declarative memory and the buffer.
- **Pattern Matching:** This mechanism selects and executes production rules by matching the current state of the buffer against the conditions stored in procedural memory, thereby determining the next attention location in the simulated scanpath

4.3 Declarative Memory Construction

The foundation of CodeACT-R is its declarative memory, which stores structured knowledge extracted from collected human scanpaths. Each statement is encoded along two dimensions: semantic properties captured through Abstract Syntax Tree (AST) analysis including semantic category (control, calculation, declaration), control level (nesting depth), and complexity (operators/function calls); and temporal position within the reading sequence using normalized values indicating early versus late viewing order. This dual encoding enables the model to learn both semantic comprehension strategies and common reading flows across different code structures.

4.4 Cross-Language Adaptation: From C++ to Java

To validate the cross-language generalizability of human attention patterns in code comprehension, we systematically adapted our CodeACT-R model to simulate scanpaths for Java programming

tasks. This adaptation process addresses whether cognitive patterns learned from one programming language can effectively transfer to enhance LLM performance on different languages.

The adaptation preserves the core ACT-R simulation architecture while implementing language-specific modifications to accommodate Java's distinct syntactic and semantic characteristics. We replaced the original C++ AST-based statement extraction with a hybrid parsing approach that combines the javalang parser for semantic analysis. This hybrid approach enables accurate identification of Java-specific constructs including object-oriented features (inheritance, polymorphism), exception handling mechanisms (try-catch blocks), and type declaration patterns. The complexity calculation component was enhanced to incorporate Java-specific operational elements while maintaining the original multi-dimensional scoring framework. Beyond the standard arithmetic operations, logical operations, function calls, and control structures, we integrated Java-specific complexity factors including instanceof operations, explicit type casting, and Java keyword density analysis. These additions ensure that the simulation accurately captures the cognitive load patterns associated with reading Java code.

This cross-language adaptation maintains the methodological integrity of the original fixation generation process while accommodating the syntactic and semantic differences between programming languages.

4.5 The Scanpath Simulation Process of CodeACT-R

The simulation simulates scanpaths through a two-step process that remains consistent across programming languages. First, CodeACT-R estimates scanpath length by comparing it against stored declarative memory to select an appropriate length. Second, the model generates each fixation using pattern-matching strategies: (1) *Semantic Pattern Matching* retrieves memory items with similar relative positions and matches semantic categories in the target code; (2) *Quick Pattern Matching* models expert shortcuts by focusing on complex statements or tracing variables backward; and (3) *Tracing Pattern Matching* models expert shortcuts by focusing on complex statements or tracing variables backward. This process operates identically whether processing C++ or Java code, with only the underlying semantic analysis adapted for language-specific constructs.

5 Methodology

Our methodology leverages simulated human attention patterns to enhance LLM performance through three key steps, as shown in Figure 4: (1) scanpath simulation from CodeACT-R and cognitive signal extraction, and also cognitively-guided fine-tune including (2) projection of these cognitive signals onto LLM subword tokens to address tokenization granularity, (3) incorporation into the training objective through scanpath-weighted loss functions for fine-tuning.

5.1 Scanpath Simulation and Signal Extraction

Given a source code snippet, we first employ CodeACT-R to simulate human visual attention behavior, yielding a scanpath—an ordered sequence of fixations representing the temporal flow of human attention during code reading. From this scanpath, we derive three aligned arrays that capture both the *coverage* and *temporal* dimensions of human attention at the statement-level code granularity.

The extraction process generates: (i) a binary attention mask $M \in \{0, 1\}^N$ indicating which of the N code tokens received human fixation, capturing the *coverage* aspect of attention; (ii) an attention order vector $O \in \mathbb{N}^N$ where O_i represents the sequential position of token i in the scanpath (with unfixed tokens assigned value 0), capturing the *temporal flow* of attention; and (iii) a serialized representation combining the raw code snippet with target output used for supervised learning. This multi-dimensional representation preserves both the selective nature of human

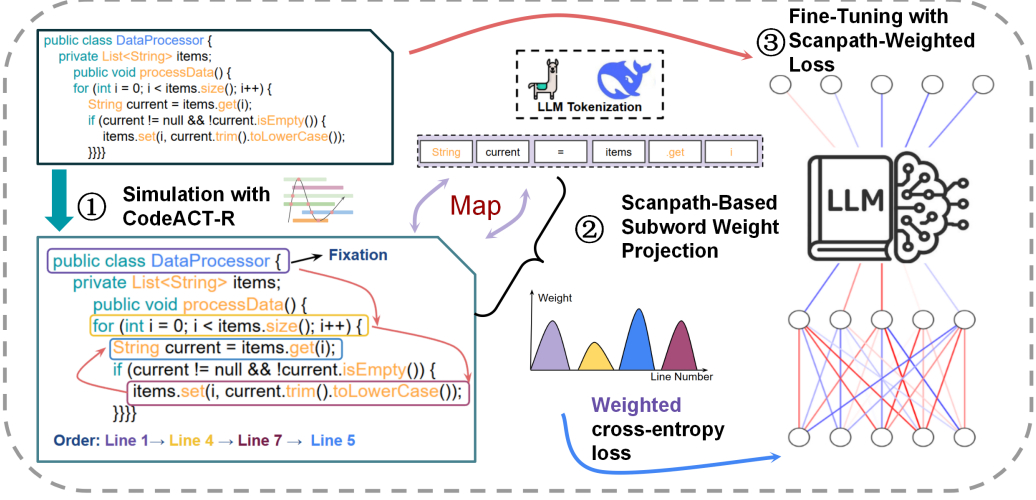


Fig. 4. The pipeline for integrating cognitive data into an LLM. (1) CodeACT-R generates a simulated scanpath from the source code. (2) The scanpath is projected onto subword tokens to create attention weights. (3) The LLM is fine-tuned using a weighted cross-entropy loss, emphasizing the tokens highlighted by the scanpath.

attention (not all code elements are equally important) and the sequential cognitive process of code comprehension.

5.2 Subword Weight Projection and Prompt Construction

A challenge in integrating cognitive signals with LLMs stems from the granularity mismatch: while human attention operates at the semantic level of complete code tokens (e.g., variable names, keywords), modern LLMs process text through subword tokenization schemes that fragment meaningful units. For instance, a variable name like `calculateSum` might be tokenized into `[calculate, Sum]`, requiring careful projection of the original cognitive signal. We address this mismatch through uniform weight propagation. For each code token T_i that the tokenizer splits into $\text{Shards}(T_i) = [s_{i,1}, \dots, s_{i,k_i}]$, we assign every shard the token's weight:

$$w_{i,j}^{\text{shard}} = w_i^{\text{scan}} \quad \text{for } j = 1, \dots, k_i.$$

Concatenating these shard weights in the same order as tokenization yields a vector $\mathbf{w}$ that is exactly as long as the model's input sequence.

We format each example with a concise instruction–output template: the prompt contains an instruction for code completion and the input snippet; the target is the desired continuation ('content'). During preprocessing we tokenize the full prompt-plus-target, mask out prompt positions in the label vector with `-100`, and attach the projected weight vector $\mathbf{w}$ to the target span only. Sequences are truncated or padded to length 1024; padding uses weight 1.0 and label `-100`. This step aligns the scanpath signal with exactly the tokens that contribute to learning.

$$\text{Shards}(T_i) = [s_{i,1}, \dots, s_{i,k_i}] \quad (1)$$

These represent the k_i subword tokens generated for scanpath token T_i . We propagate the scanpath token salience weight w_i^{scanpath} uniformly:

$$w_{i,j}^{\text{shard}} = w_i^{\text{scanpath}} \quad \text{for } j = 1, \dots, k_i. \quad (2)$$

The scanpath-based weight calculation incorporates both order and coverage information:

$$w_i^{\text{scanpath}} = w_{\text{cover}} + \frac{1}{\text{order}_i} \quad (3)$$

where w_{cover} is a constant base weight and order_i represents the sequential order of attention in the scanpath, with earlier fixations receiving higher weights.

Concatenating these shard weights in the order of the tokenized input produces

$$\mathbf{w} = [w_{1,1}^{\text{shard}}, \dots, w_{1,k_1}^{\text{shard}}, \dots, w_{M,1}^{\text{shard}}, \dots, w_{M,k_M}^{\text{shard}}]. \quad (4)$$

By construction, the length of $\mathbf{w}$ matches the input for LLM sequence exactly. During fine-tuning, each weight w_ℓ multiplies the corresponding cross-entropy term, ensuring that shards derived from high-salience scanpath tokens have proportionally greater influence on the parameter updates for model.

5.3 Fine-Tuning with a Scanpath-Weighted Loss

The right side of Figure 4 illustrates training. The integration of cognitive signals into LLM training requires modifying the standard cross-entropy objective to incorporate human attention patterns. Our approach employs a weighted loss function that amplifies the learning signal for tokens that received early or frequent human attention while maintaining the causal language modeling paradigm. Specifically, we fine-tune the causal decoder with a loss that reweights each next-token cross-entropy term by the corresponding shard weight. Let y be the target sequence and x the prompt. With labels masking the prompt, the objective is

$$\mathcal{L}_{\text{SFT}}(\phi) = -\frac{1}{N_{\text{active}}} \sum_{j=1}^L w_j \mathbb{I}[j \in \text{target}] \log P_\phi(y_j | y_{<j}, x),$$

where w_j is the projected scanpath weight for position j and N_{active} counts non-masked target tokens. Higher weights induce larger gradients, Focusing learning on tokens fixated in the scanpath.

Implementation details follow standard practice. We use sequence length 1024, AdamW with learning rate 2×10^{-5} , batch size 2 with gradient accumulation 8, and train for 3 epochs. Prompt tokens are excluded from loss via -100 labels. All experiments fix a global seed for reproducibility. We save the final checkpoint and tokenizer after training.

6 Evaluation

Our evaluation framework addresses the central question of whether cognitively-simulated human attention patterns can meaningfully enhance Large Language Model performance on software engineering tasks. This evaluation requires a multi-faceted approach that assesses both the practical utility of our ScanCoder framework and the underlying mechanisms that drive performance improvements.

We design our evaluation strategy around three key dimensions: (1) *Task Performance*: measuring improvements on established software engineering benchmarks to demonstrate practical utility; (2) *Attention Mechanism Analysis*: examining how cognitive guidance reshapes internal model representations to understand the mechanisms of improvement.

6.1 Downstream Task and Dataset

To assess whether our ScanCoder framework can enhance the performance of contemporary Large Language Models (LLMs), we conducted large-scale evaluations on two representative tasks from the CodeXGLUE benchmark [39] that capture different aspects of code understanding and generation.

Java Code Completion: This task requires models to predict missing code segments given partial context, testing local comprehension abilities, syntactic knowledge, and the capacity to maintain coherent execution flow. Code completion represents a *local attention task* where success depends primarily on understanding immediate context and following sequential logic. We use the open-source EyeTrans corpus [67], with 1,633 training, 1,005 validation, and 1,318 test examples.

Java-to-Natural Language Summarization: This task requires generating human-readable descriptions of code functionality, testing global comprehension abilities, semantic abstraction, and the capacity to identify and articulate key behavioral patterns. Summarization represents a *global attention task* where success depends on identifying semantically critical elements distributed throughout the code snippet and synthesizing them into coherent natural language descriptions. We also use the EyeTrans corpus with 16,490 training, 518 validation, and 1,095 test examples.

6.2 Metrics

We evaluated our models on two fronts: performance on downstream software engineering tasks and the analysis of the attention mechanism for LLM.

6.3 Task Performance Metrics

For the Code Completion task, we measure performance using the following scores:

- **CodeBLEU:** A specialized BLEU variant designed specifically for code evaluation that incorporates both syntactic and semantic similarity measures. CodeBLEU extends traditional BLEU by considering Abstract Syntax Tree (AST) structure matching and data flow similarity in addition to n-gram overlap [46].
- **CrystalBLEU:** An advanced BLEU-based metric specifically designed for code evaluation that enhances semantic distinguishability by intelligently filtering trivially shared n-grams while maintaining language-agnostic computational efficiency [19]. Unlike traditional BLEU, CrystalBLEU addresses the high lexical overlap inherent in programming languages, with more meaningful similarity assessments for code generation tasks.
- **Hybrid Exact Match (H-Exact):** A balanced accuracy metric calculated as $0.5 \times \text{textExactMatch} + 0.5 \times \text{textSubstringMatch}$, which considers both strict token-level matching and partial semantic overlap. This dual-component approach provides robust evaluation by rewarding both precise completions and semantically correct partial matches.

For the code summarization task, we use three standard metrics:

- **ROUGE-L:** The F1 score computed over the longest common subsequence between generated and reference summaries. ROUGE-L captures structural similarity and content ordering.
- **METEOR:** The harmonic mean of precision and recall that incorporates exact matches, stemming, and synonym detection, with additional fragmentation penalties that reward coherent, well-structured summaries.
- **BERTScore:** An F1 score derived from averaged cosine similarities of contextual token embeddings from a pretrained BERT encoder, measuring semantic overlap at the representation level rather than surface form [63].

6.4 Attention Mechanism Analysis

To understand the underlying mechanisms through which cognitive guidance improves model performance, we conduct detailed analysis of how our approach reshapes internal attention patterns. This analysis provides insights into the *how* and *why* of performance improvements, beyond black-box evaluation to mechanistic understanding.

Let $\mathbf{a} = (a_1, \dots, a_n)$ be the normalized final-layer attention over the n input tokens, C the set of semantically critical token indices, and $k = 20$. We measure:

- *Generation Confidence Score (GCS)*: The average log-likelihood of the produced sequence: $\frac{1}{T} \sum_{t=1}^T \log P(y_t | y_{<t}, x)$, where y represents the T generated tokens. Higher GCS values indicate increased model confidence in its predictions, suggesting that cognitive guidance helps models make more decisive and accurate predictions by providing clearer optimization signals.
- *Recency Focus Score (RFS)*: . The fraction of attention allocated to recent context: $\frac{\sum_{i=n-k+1}^n a_i}{\sum_{i=1}^n a_i}$. RFS near 1 reveals strong recency bias, which is beneficial for completion tasks that depend heavily on immediate syntactic context but potentially limiting for tasks requiring global understanding.
- *Average Focus Score (AFS)*: The mean attention weight on semantically critical tokens: $\frac{1}{|C|} \sum_{i \in C} a_i$. AFS quantifies how effectively the model identifies and prioritizes informationally important code elements, providing direct measurement of semantic attention alignment with human cognitive priorities.
- *Attention Entropy*: The Shannon entropy of the attention distribution: $-\sum_{i=1}^n a_i \log a_i$. Low entropy implies peaked, focused attention strategies, while high entropy indicates more distributed, exploratory processing. The optimal entropy level depends on task requirements: completion may benefit from focused attention, while summarization may require broader information integration.

6.5 Experiment Setup

Our experimental design prioritizes reproducibility and fair comparison through controlled conditions and standardized procedures. We fine-tune two open-source LLM with Transformer architectures: DeepSeek-Coder (1.3B parameters), specifically pretrained for code understanding, and Llama-3.2 (1B parameters), representing general-purpose language modeling capabilities. Each model undergoes independent training under identical hyperparameters: sequence length 1024, AdamW optimization with learning rate 2×10^{-5} , batch size 2 with gradient accumulation factor 8, and training for 3 epochs. We maintain a fixed random seed of 42 across all experiments to ensure reproducible results. All experiments are executed on a single NVIDIA A6000 GPU to ensure computational consistency, with identical preprocessing pipelines, tokenization schemes, and evaluation protocols.

7 Result

The evaluation addresses two fundamental research questions that validate both the practical utility and theoretical foundations of our approach. We organize our findings around these two research questions: (1) whether cognitive guidance from ScanCoder achieves performance improvements on established benchmarks, and (2) how our ScanCoder framework mechanically alters model behavior to achieve these improvements.

7.1 RQ1: Does cognitive guidance from ScanCoder help LLM achieve performance improvements?

To answer whether our cognitively-guided training approach improves LLM performance, we evaluate ScanCoder's impact on two downstream software engineering tasks across different model architectures. We compare baseline models fine-tuned with standard approaches against versions enhanced with our complete ScanCoder framework, which integrates simulated human attention patterns with cognitively-guided fine-tune.

Table 1. Model Performance Comparison

Metric	Llama-3.2		DeepSeek-Coder	
	Baseline	ScanCoder	Baseline	ScanCoder
<i>Completion</i>				
CodeBLEU	19.47	60.48 (+41.01)	13.80	48.78 (+34.98)
CrystalBLEU	26.65	65.90 (+39.25)	18.98	54.61 (+35.63)
H-Exact	50.85	83.38 (+32.53)	49.40	78.56 (+29.16)
<i>Summarization</i>				
METEOR	27.05	31.67 (+4.62)	22.81	32.86 (+10.05)
ROUGE-L	18.27	40.44 (+22.17)	15.12	39.84 (+24.72)
BERTScore	33.41	50.30 (+16.89)	28.64	50.57 (+21.93)

Table 1 presents comprehensive performance comparisons that reveal improvements across all evaluated configurations. The results demonstrate that ScanCoder provides benefits regardless of the underlying model architecture, suggesting that our cognitively-guided training methodology contains universally valuable signals for code understanding tasks.

7.1.1 Code Completion: For the code completion task, which requires models to predict missing code segments based on contextual understanding, ScanCoder achieves improvements across all metrics. The Llama-3.2 model enhanced with ScanCoder demonstrates performance gains: CrystalBLEU scores increase by 39.25 points (from 26.65 to 65.90). Similarly impressive, CodeBLEU scores rise by 41.01 points, while Hybrid Exact Match improvements reach 32.53 points, indicating that our cognitive guidance framework helps models generate more syntactically and semantically correct completions.

The DeepSeek-Coder model shows equally substantial improvements when enhanced with ScanCoder. CrystalBLEU gains reach 35.63 points with CodeBLEU improvements of 34.98 points and H-Exact gains of 29.16 points. These results demonstrate that even models already optimized for code tasks benefit from our cognitive guidance framework, suggesting that regular fine-tune approaches may not fully capture the nuances of human-guided code comprehension strategies. This cross-architectural effectiveness supports our hypothesis that integrating human cognitive patterns through weighted training objectives can enhance a sufficiently capable language model.

7.1.2 Code Summarization: The code summarization task, which requires models to generate natural language descriptions of code functionality, demonstrates more benefits from the ScanCoder. This task demands global comprehension abilities and semantic abstraction capabilities that align closely with the holistic attention strategies encoded in our training approach.

For Llama-3.2, ScanCoder improvements across all semantic similarity metrics: BERTScore increases by 16.89 points (from 33.41 to 50.30), while ROUGE-L scores improve by 22.17 points. METEOR scores, which incorporate linguistic sophistication through stemming and synonym matching, show gains of 4.62 points. These improvements indicate that ScanCoder helps models not only identify key semantic content but also express it more coherently and completely in natural language. The DeepSeek-Coder results with ScanCoder are more impressive, with BERTScore improvements reaching 21.93 points and ROUGE-L gains of 24.72 points. METEOR improvements of 10.05 points further confirm that our cognitive guidance framework enhances the model to generate linguistically sophisticated and semantically accurate summaries.

ScanCoder achieves performance gains across LLM architectures on code completion and summarization tasks. These consistent improvements demonstrate that cognitive guidance addresses fundamental aspects of code understanding.

Table 2. Comparison of Attention Behavior Statistics. We report the mean values for each metric across the test set. All p-values are calculated using a two-sided t-test, with values < 0.001 indicating a statistically significant difference.

Task	Metric	ScanCoder	Baseline	p-value
Completion	Generation Confidence (GCS)	-0.000	-0.035	< 0.001
	Recency Focus (RFS)	0.503	0.463	< 0.001
	— Global Attention —			
	Avg. Focus (AFS) ($\times 10^{-3}$)	9.077	9.036	0.360
	Avg. Entropy	88.545	88.511	0.936
Summarization	Generation Confidence (GCS)	-0.551	-0.470	< 0.001
	Recency Focus (RFS)	1.319	0.551	< 0.001
	— Global Attention —			
	Avg. Focus (AFS) ($\times 10^{-3}$)	7.970	3.113	< 0.001
	Avg. Entropy	62.237	24.019	< 0.001

7.2 RQ2: How ScanCoder mechanically alters LLM behavior to achieve these improvements?

To understand how ScanCoder improves LLM performance, we conduct a detailed analysis of attention mechanisms and dissect the individual contributions of different components within our cognitive guidance framework. This analysis provides insights into the mechanisms of improvement rather than simply documenting performance gains.

7.2.1 LLM Attention Mechanism Reshaping Through Cognitively-Guided Fine-tune. Our mechanistic analysis reveals that ScanCoder alters model attention patterns in ways that differ dramatically between tasks, as demonstrated by the quantitative statistics in Table 2 and confirmed by attention visualization in Figure 5. These changes demonstrate how our approach transfers human cognitive strategies to machine learning models.

Code Completion: For code completion tasks, models trained with ScanCoder show higher Generation Confidence Scores (GCS) and Recency Focus Scores (RFS) compared to baseline models, with both differences achieving statistical significance ($p < 0.001$). The improved GCS indicates that our cognitive guidance framework helps models make more confident predictions by emphasizing learning signals that align with human attention priorities. The higher RFS demonstrates that ScanCoder reinforces appropriate sequential processing strategies, where recent context receives prioritized attention for predicting subsequent code elements.

Critically, the global attention metrics (Average Focus Score and Attention Entropy) show no significant differences for completion tasks, indicating that cognitive guidance in ScanCoder

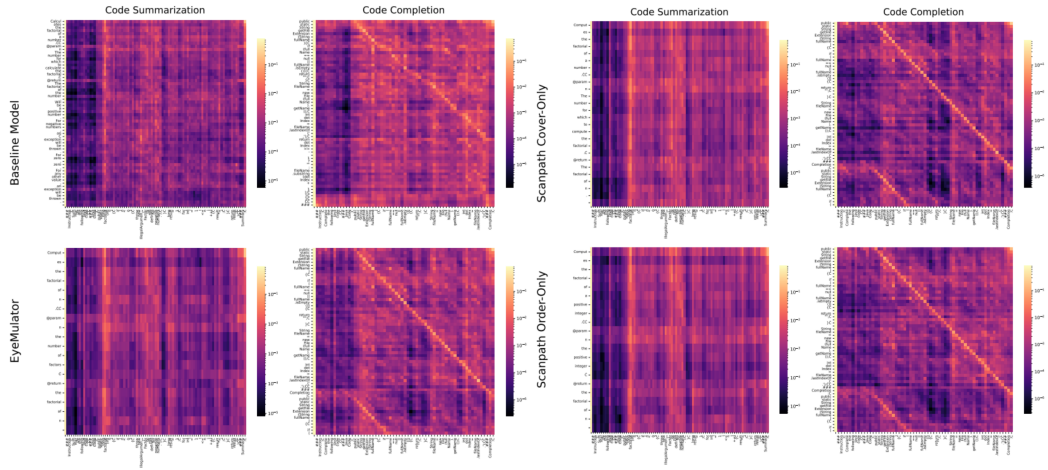


Fig. 5. Attention mechanism comparison between baseline models and ScanCoder across different software engineering tasks. The heatmaps visualize how models attend to input code tokens (x-axis) when generating output tokens (y-axis), revealing attention strategies learned through cognitive guidance.

primarily enhances local processing strategies rather than altering global attention distribution. This demonstrates the precision of our approach—cognitive guidance adapts to task requirements rather than imposing uniform attention changes. The attention heatmaps corroborate these quantitative findings, showing strong diagonal patterns in ScanCoder-trained models that indicate focused recency bias.

Code Summarization: In contrast, code summarization demonstrates comprehensive changes across all attention metrics when trained with ScanCoder, indicating that our framework transforms how models process code for global understanding tasks.

Most notably, ScanCoder-trained models exhibit $2.5\times$ higher Average Focus Scores on semantically critical tokens, indicating improved ability to identify and prioritize informationally important code elements throughout the entire snippet. This enhanced semantic focus represents a shift enabled by our approach, transforming diffuse attention patterns into strategic, human-like identification of key functional components.

The $2.6\times$ higher Attention Entropy reveals adoption of more distributed, exploratory attention strategies that consider broader ranges of information when generating summaries. This increased entropy indicates that ScanCoder teaches models to adopt holistic comprehension strategies that mirror human cognitive approaches to understanding code functionality.

The attention visualizations in figure 5 illustrate transformative effect from ScanCoder: baseline models show diffuse, seemingly random attention patterns, while ScanCoder-trained models exhibit distinct vertical stripes representing focused attention on semantically critical anchor points distributed throughout the code.

ScanCoder alters attention patterns in heatmap differently for each task. Code completion shows enhanced local focus with higher confidence scores, while summarization exhibits transformed global understanding with $2.5\times$ higher focus on semantically critical tokens and $2.6\times$ higher attention entropy.

Table 3. Key metric comparison on Human Attention Pattern Decomposition. Coverage-Only uses only the binary attention mask indicating which tokens received human fixation, while Order-Only uses only the temporal sequence information of attention flow. ScanCoder combines both components through weighted training approach.

Task (Metric)	Baseline	Order-Only	Coverage-Only	ScanCoder
Summarization (BERTScore)	33.41	46.37 (+38.79%)	46.88 (+40.32%)	50.30 (+50.55%)
Completion (HybridExact)	50.85	83.13 (+63.48%)	81.09 (+59.47%)	83.38 (+63.97%)

7.2.2 *Human Attention Pattern Decomposition.* As presented in Table 3, we provide insights into the relative importance of different components within cognitive guidance from ScanCoder. By isolating coverage information (which tokens receive attention) from order information (the temporal sequence of attention), we identify how different aspects of our approach contribute to model improvements.

The results reveal task-dependent patterns in the relative importance of different components within ScanCoder. For code summarization, the coverage-only variant achieves BERTScore improvements of 40.32% over baseline, exceeding the 38.79% improvement from the order-only variant. This pattern suggests that for global comprehension tasks, coverage attention guidance from ScanCoder (knowing *what* to attend to) provides the primary performance benefit, while temporal sequencing plays a supporting role.

Conversely, for code completion, the order-only variant yields 63.48% improvement in HybridExact scores, notably higher than the 59.47% gain from coverage-only guidance. This reversal indicates that for sequential prediction tasks, temporal attention guidance from ScanCoder (the *sequence* of information processing) is more critical than simply identifying important locations. This finding validates our ScanCoder to incorporate both coverage and temporal dimensions of human attention.

For both tasks, the full model ('ScanCoder') outperforms both partial variants, achieving the highest scores of 50.30 on summarization and 83.38 on completion. The Scanpath provides complementary, non-redundant information; the model performs best when it is guided by both the important locations in the code and the human-like sequence for processing them.

Study on human attention pattern decomposition reveals task-dependent importance: temporal order information dominates for completion tasks, while coverage slightly leads for summarization. The complete ScanCoder framework outperforms both partial variants, demonstrating synergistic cognitive signal integration.

8 Implications and Discussion

Our study demonstrates the first application of cognitive simulation to enhance LLM performance for software engineering tasks. It provides insights into broader implications for cognitive modeling and human-LLM collaboration in software development.

8.1 Cross-Domain Generalizability for Cognitive Simulation

The success in cross-language transfer from C++ cognitive patterns to Java programming tasks establishes empirical evidence for the universality of human cognitive strategies in code comprehension. This generalizability extends beyond programming languages to suggest broader applicability across software engineering domains. The cognitive patterns captured through CodeACT-R simulation reflect fundamental human problem-solving strategies that transcend specific syntactic constructs, indicating potential for adaptation to diverse software engineering activities, including requirements analysis, system design, and code review processes.

The ability of framework to maintain semantic coherence while adapting to different programming paradigms demonstrates that cognitive modeling can capture abstract reasoning patterns rather than surface-level behaviors. This finding encourages the development of specialized cognitive models tailored to various software engineering tasks, where simulated human expertise can provide valuable guidance for AI systems across the software development lifecycle. Such cognitive insights can prove invaluable for educational contexts, enabling more effective teaching strategies that scaffold novice programmers by modeling expert cognitive approaches to complex software engineering challenges.

8.2 Scalable Human-AI Training Paradigms

The ScanCoder framework addresses scalability constraints in human-AI collaboration by reducing dependency on resource-intensive empirical studies while maintaining cognitive authenticity. ScanCoder generates extensive cognitive data from minimal initial human observations, enabling LLM training with human-informed guidance. This scalability transformation facilitates automated generation of high-quality cognitive datasets, allowing researchers to train LLM models on diverse cognitive behaviors without repeated human studies. The approach enables faster and more efficient data collection for human-AI collaboration research, supporting the development of sophisticated tools for automated code generation, error prediction, and personalized learning environments where LLM adapt recommendations based on inferred cognitive states.

8.3 Programming Paradigm Expansion

While our current evaluation focuses on object-oriented, imperative languages (C++ and Java), the success of cross-language transfer suggests promising opportunities for extending cognitive simulation to diverse programming paradigms. Functional programming languages like Haskell and OCaml present intriguing challenges for cognitive modeling, as they require fundamentally different attention strategies emphasizing mathematical reasoning, type inference, and compositional thinking over sequential control flow. Declarative programming paradigms offer another frontier, where cognitive patterns must capture set-theoretic operations, relational algebra, and constraint satisfaction reasoning rather than procedural logic. Database query languages such as SQL represent particularly compelling targets, as they demand specialized attention patterns involving extensive cross-referencing between schemas, join conditions, and aggregate operations. The abstract syntactic analysis foundation of ScanCoder positions it well for adaptation to these diverse paradigms, suggesting that cognitive simulation could provide valuable insights across the full spectrum of programming languages and computational thinking approaches.

9 Limitation

While ScanCoder demonstrates promising results for cognitively-guided LLM enhancement, several limitations should also be considered.

Individual Cognitive Variation. Human programmers exhibit individual differences in attention strategies based on expertise level, programming experience, and cognitive style, potentially limiting our approach, which learns average patterns from collected data. We mitigate this concern through our coverage metric evaluation, which demonstrates that our simulation captures diverse attention strategies rather than just average behavior. However, future work could investigate personalized cognitive guidance based on individual programmer profiles.

Task-Specific Evaluation Scope. While we demonstrate effectiveness on code completion and summarization tasks, broader applicability to other software engineering activities remains empirically unvalidated. We mitigate this through our task-dependent attention analysis, revealing how cognitive guidance adapts differently for local versus global comprehension requirements. This mechanistic understanding suggests potential for extension to related tasks such as code review, debugging, and requirements analysis, though direct evaluation would strengthen these claims.

10 Conclusion

We present ScanCoder, the first comprehensive framework that integrates cognitive simulation through ACT-R architecture with Large Language Model enhancement for software engineering tasks. Our approach addresses limitations in human-LLM collaboration by demonstrating that simulated human attention patterns can effectively guide LLM training at scale. Through extensive evaluation, we achieve performance improvements—up to 39 points on CrystalBLEU for code completion and 22 points on BERTScore for summarization—while establishing cross-language generalizability from C++ cognitive data to Java programming tasks. Our mechanistic analysis reveals that cognitive guidance fundamentally reshapes model attention in task-dependent ways: reinforcing local focus for completion tasks while fostering holistic comprehension strategies for summarization, increasing attention on semantically critical tokens by $2.5\times$.

This work offers both theoretical insights into human-machine alignment and practical solutions for resource-constrained cognitive data collection. By demonstrating that cognitively-simulated attention patterns contain universally valuable signals that transcend specific programming languages and model architectures, our framework opens promising avenues for scalable human-guided LLM training. Future research can build upon our foundation to explore personalized cognitive guidance, extend to additional software engineering tasks, and investigate the broader applicability of cognitive simulation in human-LLM collaboration across diverse domains beyond programming.

11 Data Availability

We share the result and implementation code of this study¹. We will also make data publicly available upon acceptance.

References

- [1] Deepak Bhaskar Acharya, Karthigeyan Kuppan, and B Divya. 2025. Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey. *IEEE Access* (2025).
- [2] Khlood Ahmad, Mohamed Abdelrazek, Chetan Arora, Arbind Agrahari Baniya, Muneera Bano, and John Grundy. 2023. Requirements engineering framework for human-centered artificial intelligence software systems. *Applied Soft Computing* 143 (2023), 110455.
- [3] Maike Ahrens, Kurt Schneider, and Melanie Busch. 2019. Attention in software maintenance: an eye tracking study. In *2019 IEEE/ACM 6th International Workshop on Eye Movements in Programming (EMIP)*. IEEE, 2–9.
- [4] Tarek Alakmeh, David Reich, Lena Jäger, and Thomas Fritz. 2024. Predicting Code Comprehension: A Novel Approach to Align Human Gaze with Code using Deep Neural Networks. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1982–2004.
- [5] John R Anderson. 1996. ACT: A simple theory of complex cognition. *American psychologist* 51, 4 (1996), 355.

¹<https://anonymous.4open.science/r/ScanCoder-706D>

- [6] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862* (2022).
- [7] Aakash Bansal, Chia-Yi Su, Zachary Karas, Yifan Zhang, Yu Huang, Toby Jia-Jun Li, and Collin McMillan. 2023. Modeling programmer attention as scanpath prediction. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1732–1736.
- [8] Teresa Busjahn and Carsten Schulte. 2013. The use of code reading in teaching programming. In *Proceedings of the 13th Koli Calling international conference on computing education research*. 3–11.
- [9] Teresa Busjahn, Carsten Schulte, and Andreas Busjahn. 2011. Analysis of code reading to gain more insight in program comprehension. In *Proceedings of the 11th Koli Calling International Conference on Computing Education Research*. 1–9.
- [10] T. Busjahn, Carsten Schulte, Bonita Sharif, Simon, Andrew Begel, M. Hansen, R. Bednarik, P. Orlov, Petri Ihantola, G. Shchekotova, and M. Antropova. 2014. Eye tracking in computing education. (2014), 3–10. <https://doi.org/10.1145/2632320.2632344>
- [11] Michael D Byrne. 2001. ACT-R/PM and menu selection: Applying a cognitive architecture to HCI. *International Journal of Human-Computer Studies* 55, 1 (2001), 41–84.
- [12] Tara Capel and Margot Brereton. 2023. What is human-centered about human-centered AI? A map of the research landscape. In *Proceedings of the 2023 CHI conference on human factors in computing systems*. 1–23.
- [13] Shreyas Chaudhari, Pranjal Aggarwal, Vishvak Murahari, Tanmay Rajpurohit, Ashwin Kalyan, Karthik Narasimhan, Ameet Deshpande, and Bruno Castro da Silva. 2024. Rlhf deciphered: A critical analysis of reinforcement learning from human feedback for llms. *Comput. Surveys* (2024).
- [14] Hongpeng Chen, Shufei Li, Junming Fan, Anqing Duan, Chenguang Yang, David Navarro-Alarcon, and Pai Zheng. 2025. Human-in-the-loop robot learning for smart manufacturing: A human-centric perspective. *IEEE Transactions on Automation Science and Engineering* (2025).
- [15] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep reinforcement learning from human preferences. *Advances in neural information processing systems* 30 (2017).
- [16] Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. 2023. Safe rlhf: Safe reinforcement learning from human feedback. *arXiv preprint arXiv:2310.12773* (2023).
- [17] Dejian Yang Zhenda Xie Kai Dong Wentao Zhang Guanting Chen Xiao Bi Y. Wu Y.K. Li Fuli Luo Yingfei Xiong Wenfeng Liang Daya Guo, Qihao Zhu. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. <https://arxiv.org/abs/2401.14196>
- [18] Hanze Dong, Wei Xiong, Bo Pang, Haoxiang Wang, Han Zhao, Yingbo Zhou, Nan Jiang, Doyen Sahoo, Caiming Xiong, and Tong Zhang. 2024. Rlhf workflow: From reward modeling to online rlhf. *arXiv preprint arXiv:2405.07863* (2024).
- [19] Aryaz Eghbali and Michael Pradel. 2022. CrystalBLEU: precisely and efficiently measuring the similarity of code. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.
- [20] Aryaz Eghbali and Michael Pradel. 2024. De-Hallucinator: Mitigating LLM Hallucinations in Code Generation Tasks via Iterative Grounding. *arXiv preprint arXiv:2401.01701* (2024).
- [21] Yousef Emami, Luis Almeida, Kai Li, Wei Ni, and Zhu Han. 2024. Human-in-the-loop machine learning for safe and ethical autonomous vehicles: Principles, challenges, and opportunities. *arXiv preprint arXiv:2408.12548* (2024).
- [22] Helen J. A. Fuller, M. Reed, and Yili Liu. 2012. Integration of Physical and Cognitive Human Models to Simulate Driving With a Secondary In-Vehicle Task. *IEEE Transactions on Intelligent Transportation Systems* 13 (2012), 967–972. <https://doi.org/10.1109/TITS.2012.2182764>
- [23] Drew T Guarnera, Corey A Bryant, Ashwin Mishra, Jonathan I Maletic, and Bonita Sharif. 2018. itrace: Eye tracking infrastructure for development environments. In *Proceedings of the 2018 ACM symposium on eye tracking research & applications*. 1–3.
- [24] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. Unixcoder: Unified cross-modal pre-training for code representation. *arXiv preprint arXiv:2203.03850* (2022).
- [25] Desta Haileselassie Hagos, Rick Battle, and Danda B Rawat. 2024. Recent advances in generative ai and large language models: Current status, challenges, and perspectives. *IEEE transactions on artificial intelligence* (2024).
- [26] Kenneth Holmqvist, Marcus Nyström, Richard Andersson, Richard Dewhurst, Halszka Jarodzka, and Joost Van de Weijer. 2011. *Eye tracking: A comprehensive guide to methods and measures*. oup Oxford.
- [27] Anthony J Hornof and David E Kieras. 1997. Cognitive modeling reveals menu search in both random and systematic. In *Proceedings of the ACM SIGCHI Conference on Human factors in computing systems*. 107–114.
- [28] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2. 5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).
- [29] Liu Jiang, Shixia Liu, and Changjian Chen. 2019. Recent research advances on interactive machine learning. *Journal of Visualization* 22, 2 (2019), 401–417.

- [30] Zachary Karas, Aakash Bansal, Yifan Zhang, Toby Li, Collin McMillan, and Yu Huang. 2024. A Tale of Two Comprehensions? Analyzing Student Programmer Attention during Code Summarization. *ACM Transactions on Software Engineering and Methodology* (2024).
- [31] Timo Kaufmann, Paul Weng, Viktor Bengs, and Eyke Hüllermeier. 2024. A survey of reinforcement learning from human feedback. (2024).
- [32] Ksenia Konyushkova, Raphael Sznitman, and Pascal Fua. 2017. Learning active learning from data. *Advances in neural information processing systems* 30 (2017).
- [33] Kyle Krafka, Aditya Khosla, Petr Kellnhofer, Harini Kannan, Suchendra Bhandarkar, Wojciech Matusik, and Antonio Torralba. 2016. Eye tracking for everyone. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2176–2184.
- [34] Sushant Kumar, Sumit Datta, Vishakha Singh, Deepanwita Datta, Sanjay Kumar Singh, and Ritesh Sharma. 2024. Applications, challenges, and future directions of human-in-the-loop learning. *IEEE Access* 12 (2024), 75735–75760.
- [35] Guangliang Li, Randy Gomez, Keisuke Nakamura, and Bo He. 2019. Human-centered reinforcement learning: A survey. *IEEE Transactions on Human-Machine Systems* 49, 4 (2019), 337–349.
- [36] Jiliang Li, Yifan Zhang, Zachary Karas, Collin McMillan, Kevin Leach, and Yu Huang. 2024. Do machines and humans focus on similar code? exploring explainability of large language models in code summarization. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*. 47–51.
- [37] Yu-Tzu Lin, Cheng-Chih Wu, Ting-Yun Hou, Yu-Chih Lin, Fang-Ying Yang, and Chia-Hu Chang. 2015. Tracking students' cognitive processes during program debugging—An eye-movement approach. *IEEE transactions on education* 59, 3 (2015), 175–186.
- [38] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. 2024. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971* (2024).
- [39] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. *CoRR* abs/2102.04664 (2021).
- [40] Ariana Martino, Michael Iannelli, and Coleen Truong. 2023. Knowledge injection to counter large language model (LLM) hallucination. In *European Semantic Web Conference*. Springer, 182–185.
- [41] Agnieszka Mikołajczyk and Michał Grochowski. 2018. Data augmentation for improving deep learning in image classification problem. In *2018 international interdisciplinary PhD workshop (IIPhDW)*. IEEE, 117–122.
- [42] Eduardo Mosqueira-Rey, Elena Hernández-Pereira, David Alonso-Ríos, José Bobes-Bascarán, and Ángel Fernández-Leal. 2023. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review* 56, 4 (2023), 3005–3054.
- [43] U. Obaidallah, Mohammed Al Haek, and P. Cheng. 2018. A Survey on the Usage of Eye-Tracking in Computer Programming. *ACM Computing Surveys (CSUR)* 51 (2018), 1 – 58. <https://doi.org/10.1145/3145904>
- [44] David N Palacio, Daniel Rodriguez-Cardenas, Alejandro Velasco, Dipin Khati, Kevin Moran, and Denys Poshyvanyk. 2024. Towards more trustworthy and interpretable LLMs for code through syntax-grounded explanations. *arXiv preprint arXiv:2407.08983* (2024).
- [45] Norman Peitek, Janet Siegmund, and Sven Apel. 2020. What drives the reading order of programmers? an eye tracking study. In *Proceedings of the 28th International Conference on Program Comprehension*. 342–353.
- [46] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).
- [47] F. Ritter, G. Baxter, Gary Jones, and R. M. Young. 2000. Supporting cognitive models as users. *ACM Trans. Comput. Hum. Interact.* 7 (2000), 141–173. <https://doi.org/10.1145/353485.353486>
- [48] Frank E Ritter, Farnaz Tehrani, and Jacob D Oury. 2019. ACT-R: A cognitive architecture for modeling cognition. *Wiley Interdisciplinary Reviews: Cognitive Science* 10, 3 (2019), e1488.
- [49] Daniel Rodriguez-Cardenas. 2025. Towards More Interpretable Large Language Models for Code. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 1270–1272.
- [50] Dario D Salvucci. 2006. Modeling driver behavior in a cognitive architecture. *Human factors* 48, 2 (2006), 362–380.
- [51] Tim Schürmann and Philipp Beckerle. 2020. Personalizing human-agent interaction through cognitive models. *Frontiers in Psychology* 11 (2020), 561510.
- [52] Shreya Shankar, JD Zamfirescu-Pereira, Björn Hartmann, Aditya Parameswaran, and Ian Arawjo. 2024. Who validates the validators? aligning llm-assisted evaluation of llm outputs with human preferences. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–14.
- [53] Zohreh Sharafi, Bonita Sharif, Yann-Gaël Guéhéneuc, Andrew Begel, Roman Bednarik, and Martha Crosby. 2020. A practical guide on conducting eye tracking studies in software engineering. *Empirical Software Engineering* 25 (2020), 3128–3174.

- [54] Zohreh Sharafi, Z. Soh, and Yann-Gaël Guéhéneuc. 2015. A systematic literature review on the usage of eye-tracking in software engineering. *Inf. Softw. Technol.* 67 (2015), 79–107. <https://doi.org/10.1016/j.infsof.2015.06.008>
- [55] Wei Shen, Guanlin Liu, Zheng Wu, Ruofei Zhu, Qingping Yang, Chao Xin, Yu Yue, and Lin Yan. 2025. Exploring data scaling trends and effects in reinforcement learning from human feedback. *arXiv preprint arXiv:2503.22230* (2025).
- [56] Chandan Singh, Jeevana Priya Inala, Michel Galley, Rich Caruana, and Jianfeng Gao. 2024. Rethinking interpretability in the era of large language models. *arXiv preprint arXiv:2402.01761* (2024).
- [57] Patrick Slade, Christopher Atkeson, J Maxwell Donelan, Han Houdijk, Kimberly A Ingraham, Myunghee Kim, Kyoungchul Kong, Katherine L Poggensee, Robert Riener, Martin Steinert, et al. 2024. On human-in-the-loop optimization of human–robot interaction. *Nature* 633, 8031 (2024), 779–788.
- [58] Qwen Team. 2025. Qwen3 Technical Report. *arXiv:2505.09388* [cs.CL] <https://arxiv.org/abs/2505.09388>
- [59] Xinru Wang, Hannah Kim, Sajjadur Rahman, Kushan Mitra, and Zhengjie Miao. 2024. Human-llm collaborative annotation through effective verification of llm labels. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–21.
- [60] Jia-Yu Yao, Kun-Peng Ning, Zhen-Hui Liu, Mu-Nan Ning, Yu-Yang Liu, and Li Yuan. 2023. Llm lies: Hallucinations are not bugs, but features as adversarial examples. *arXiv preprint arXiv:2310.01469* (2023).
- [61] Chenlu Ye, Wei Xiong, Yuheng Zhang, Hanze Dong, Nan Jiang, and Tong Zhang. 2024. Online iterative reinforcement learning from human feedback with general preference model. *Advances in Neural Information Processing Systems* 37 (2024), 81773–81807.
- [62] Chao Zhang, Shihan Wang, Henk Aarts, and Mehdi Dastani. 2021. Using cognitive models to train warm start reinforcement learning agents for human-computer interactions. *arXiv preprint arXiv:2103.06160* (2021).
- [63] Tianyi Zhang*, Varsha Kishore*, Felix Wu*, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=SkeHuCVFDr>
- [64] Yueke Zhang, Zihan Fang, Greg Trafton, Daniel Levin, Kevin Leach, and Yu Huang. 2025. CodeACT-R: A Cognitive Simulation Framework for Human Attention in Code Reading. In *Proceedings of 40th IEEE/ACM International Conference on Automated Software Engineering, Nier Paper*. 456–460.
- [65] Yifan Zhang, Chen Huang, Zachary Karas, Thuy Dung Nguyen, Kevin Leach, and Yu Huang. 2025. Enhancing code llm training with programmer attention. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 616–620.
- [66] Yifan Zhang, Chen Huang, Yueke Zhang, Jiahao Zhang, Toby Jia-Jun Li, Collin McMillan, Kevin Leach, and Yu Huang. 2025. EyeMulator: Improving Code Language Models by Mimicking Human Visual Attention. *arXiv preprint arXiv:2508.16771* (2025).
- [67] Yifan Zhang, Jiliang Li, Zachary Karas, Aakash Bansal, Toby Jia-Jun Li, Collin McMillan, Kevin Leach, and Yu Huang. 2024. EyeTrans: Merging Human and Machine Attention for Neural Code Summarization. *arXiv preprint arXiv:2402.14096* (2024).
- [68] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 481–503.